Termes, substitutions, filtrage et unification

Introduction

Le but du TP est de programmer les algorithmes de filtrage et d'unification sur des termes du premier ordre. Dans un premier temps, on définit une représentation des termes et des substitutions, ainsi que des opérations associées (appliquer et composer des substitutions).

Pour chaque question, vous aurez très certainement besoin de définir quelques fonctions auxiliaires à la définition de la fonction demandée. Vous pouvez *toujours* les programmer vous-même (le choix le plus formateur), vous pouvez *quelquefois* les trouver dans une bibliothèque d'OCaml.

Le sujet

On se donne pour représentation des termes les éléments du type suivant :

```
type term =
  | Var of string
  | Term of string * term list
```

À l'aide de ce type, on représentera par exemple :

la variable x	par la valeur	`Var "x"`,
la constante `Zero`	par l'expression	`Term ("Zero", [ ])`,
le terme `Succ (Zero)`	par	`Term ("Succ", [Term ("Zero", [ ])])`.

Bien sûr, il ne faudra pas hésiter à user de définitions comme :

```
let zero = Term ("Zero", [ ]) ;;
let succ t = Term ("Succ", [t]) ;;
let plus (t1,t2) = Term ("Plus", [t1; t2]) ;;
let x = Var "x" and y = Var "y" and z = Var "z" ;;
```

pour pouvoir ensuite construire des termes plus complexes comme :

```
# plus (succ zero, plus (zero, succ zero)) ;;
- : term =
Term ("Plus",
 [Term ("Succ", [Term ("Zero", [ ] )]);
  Term ("Plus", [Term ("Zero", [ ] ); Term ("Succ", [Term ("Zero", [ ] )])])])
```

Enfin, il est plus aisé de réaliser ces exercices dans une boucle interactive OCaml plutôt que dans des fichiers exécutables. Il est donc conseillé de copier-coller vos définitions depuis votre éditeur de texte favori vers une boucle OCaml.

Sinon, vous pouvez charger un fichier `.ml` directement dans la boucle interactive d'OCaml en utilisant la directive `#use`. Ainsi, si votre fichier s'appelle `terms.ml` vous pouvez invoquer cette directive comme suit.

```
$ ocaml
        OCaml version 4.07.1

# #use "terms.ml" ;;
type term = Var of string | Term of string * term list
val vars_term : term -> string list -> string list = <fun>
val vars_term_list : term list -> string list -> string list = <fun>
...
```

1 Travail à réaliser

1.1 Termes

Q1 Définir la fonction `vars` : `term` → `string list` qui calcule la liste des variables apparaissant dans un terme.

Une possibilité consiste à définir dans un premier temps deux fonctions récursives :

`vars_term` : `term` → `string list` → `string list`

qui accumule dans son second argument la liste des variables qu'elle rencontre dans le premier, et

`vars_term_list` : `term list` → `string list` → `string list`

qui itère le calcul de `vars_term` *sur une liste de termes.*

L'utilisation d'un accumulateur permettra d'éviter d'utiliser des concaténations de listes (`@`) au profit d'ajouts en tête (`::`), cette dernière opération étant plus efficace.

1.2 Substitutions

On représentera les substitutions par des listes de paires dont la première composante est une chaîne (le nom de la variable) et dont la seconde est un terme (l'image de cette variable).

On pourra par exemple définir :

`type subst` = (`string` $*$ `term`) `list` ;;

de sorte que le type `subst` soit équivalent à (`string` $*$ `term`) `list`.

Q2 Définir une fonction `image` : `string` $\rightarrow$ `subst` $\rightarrow$ `term` qui calcule l'image d'un nom de variable par une substitution. On considérera que (`image s` θ) vaut θ(`s`) si `s` est dans le domaine de θ, et que (`image s` θ) vaut `Var s` sinon.

Q3 Définir une fonction `app_subst` : `subst` $\rightarrow$ `term` $\rightarrow$ `term`, qui calcule le transformé d'un terme par une substitution.

On pourra utiliser la fonction

`List.map` : $(\alpha \rightarrow \beta) \rightarrow \alpha$ `list` $\rightarrow \beta$ `list`

qui est telle que :

`List.map f` [$\mathtt{e}_1$; ...; $\mathtt{e}_n$] = [$\mathtt{f}(\mathtt{e}_1)$; ...; $\mathtt{f}(\mathtt{e}_n)$]

Q4 Définir `comp_subst` θ_2 θ_1 : `subst` $\rightarrow$ `subst` $\rightarrow$ `subst`, qui calcule la composée de deux substitutions (donc $\theta_2 \circ \theta_1$).

Notez qu'étant données deux substitutions θ_1 et θ_2, le domaine de $\theta_2 \circ \theta_1$ peut être décomposé en la réunion des deux ensembles disjoints $\mathsf{dom}(\theta_1)$ et $\mathsf{dom}(\theta_2) \backslash \mathsf{dom}(\theta_1)$.

Ainsi, la représentation de la substitution $\theta_2 \circ \theta_1$ peut être construite en concaténant deux listes correspondant à chacun des deux ensembles ci-dessus.

Il faut donc calculer $\theta_2(\theta_1)$ (dont le domaine est bien celui de θ_1) et ne conserver de θ_2 que les liaisons pour des variables n'appartenant pas au domaine de θ_1.

1.3 Filtrage et unification

On notera, pour les deux questions suivantes, l'existence de la fonction :

`List.combine` : α `list` $\rightarrow \beta$ `list` $\rightarrow (\alpha * \beta)$ `list`

qui est telle que :

`List.combine` [$\mathtt{x}_1$; ...; $\mathtt{x}_n$] [$\mathtt{y}_1$; ...; $\mathtt{y}_n$] = [($\mathtt{x}_1$, $\mathtt{y}_1$); ...; ($\mathtt{x}_n$, $\mathtt{y}_n$)]

et qui échoue si les deux listes sont de longueurs différentes.

Q5 Définir la fonction `filtre` : `term` $\rightarrow$ `term` $\rightarrow$ `subst` de filtrage d'un terme par un motif. Cette fonction lèvera une exception lorsque le second terme n'est pas une instance du premier, et aura une forme similaire à celle présentée dans le cours, qui reçoit une liste de couples de termes.

Q6 Définir la fonction `unifie` : `term` $\rightarrow$ `term` $\rightarrow$ `subst` qui calcule, quand c'est possible, l'unificateur le plus général de deux termes. On programmera une fonction auxiliaire

`occurs` : `string` $\rightarrow$ `term` $\rightarrow$ `bool`

qui teste si son premier argument (un nom de variable) apparaît dans le second (un terme). Cette fonction pourra utiliser la fonction prédéfinie

`List.exists` : $(\alpha \rightarrow$ `bool`$) \rightarrow \alpha$ `list` $\rightarrow$ `bool`

telle que `List.exists p` $[\mathtt{x}_1;\ ...;\ \mathtt{x}_n]$ vaut `true` si il existe un $\mathtt{x}_i$ tel que $\mathtt{p}(\mathtt{x}_i)$ est vrai, et `false` dans le cas contraire.

1.4 S'il vous reste du temps ...

... définissez une fonction `print` qui reçoit un terme, et l'imprime dans la sortie standard. Vous utiliserez la fonction `Printf.printf`.